Chapter 5 Insertion sort and introduction to complexity

Adding Elements to a List (Python Lists / Dynamic Arrays)

Prompt 1: What does "adding elements to a list" mean in Python?

Answer:

Adding an element to a Python list means:

- Placing a new value into the list
- Possibly **resizing** the underlying array
- Possibly **shifting existing elements** to make room

Python lists are implemented as **dynamic arrays**, so insertions are not always constant time.

Prompt 2: What are the main ways to add elements to a Python list?

Answer:

There are two primary methods:

1. `append(value)` → add element at the **end**
2. `insert(k, value)` → add element at **index k**

Each has different performance costs.

Prompt 3: How does `append(value)` work?

Answer:

- Adds an element to the **end of the list**
- If there is unused capacity, insertion is immediate
- If capacity is full, the array is **resized (usually doubled)**

Time Complexity:

- $O(1)$

📌 Resizing happens rarely, so average cost per operation is constant.

Prompt 4: What does `insert(k, value)` do?

Answer:

- Inserts `value` at index `k`, where `0 ≤ k ≤ n`
- All elements from index `k` onward must be **shifted right**
- May also require resizing

Prompt 5: What are the two costs involved in `insert(k, value)`?

Answer:

There are **two separate costs**:

1. **Resizing cost**
    - Happens only if array is full
    - Amortized: O(1)
2. **Shifting cost**
    - Elements from index `k` to `n-1` must shift right
    - Cost depends on `k`

Prompt 6: How does the shifting process work?

Answer (ASCII Diagram – Figure 5.16 style):

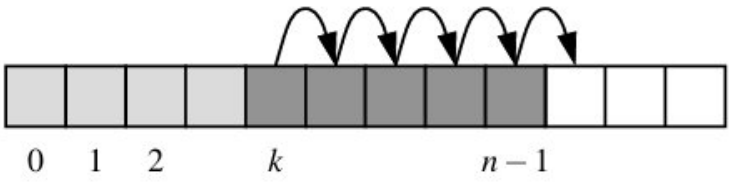

Figure 5.16: Creating room to insert a new element at index k of a dynamic array.

Prompt 7: Why do we shift from right to left?

Answer:

Shifting from right to left prevents overwriting elements before they are copied.

This ensures correctness.

Prompt 8: What is the time complexity of `insert(k, value)`?

Answer:

Let `n` be the current list size.

- Number of shifts = `n - k`
- Shifting cost = `O(n - k)`
- Resizing cost = `O(1)` amortized

Overall amortized complexity:

$$O(n-k+1)$$

Prompt 9: Why is inserting at the end efficient?

Answer:

- No shifting required
- Equivalent to `append`
- Only resizing (rare) costs time

$$O(1)\ amortized$$

Prompt 10: What experiment was performed to measure insert efficiency?

Answer:

Three insertion patterns were tested:

Case 1: Insert at beginning

```
data.insert(0, None)
```

Case 2: Insert in the middle

```
data.insert(n // 2, None)
```

Case 3: Insert at the end

```
data.insert(n, None)
```

Prompt 11: How do the measured times grow as n increases?

Answer:

- Insert at front and middle:
    - Time grows **linearly** with `n`
- Insert at end:
    - Time remains **nearly constant**

Removing Elements from a List (Python Lists / Dynamic Arrays)

Prompt 1: What does it mean to remove an element from a list?

Answer:

Removing an element from a Python list means:

- Deleting one element
- Preserving the **relative order** of the remaining elements
- Possibly **shifting elements** to fill the gap

Because Python lists are backed by **dynamic arrays**, removing elements is not always constant time.

Prompt 2: What are the main ways to remove elements from a Python list?

Answer:

Python's `list` class provides three main removal methods:

1. `pop()` – removes the **last element**
2. `pop(k)` – removes the element at **index k**
3. `remove(value)` – removes the **first occurrence of a value**

Each has **different performance characteristics**.

Prompt 3: How does `pop()` (without arguments) work?

Answer:

- `pop()` removes the **last element** of the list.
- No shifting of elements is required.
- All other elements remain in their original positions.

Example:

```
A = [10, 20, 30, 40]

A.pop()   # removes 40
```

Result:

```
[10, 20, 30]
```

Time Complexity:

$$O(1)(amortized)$$

📌 **Why amortized?**
Occasionally, Python may resize (shrink) the underlying array to save memory, but this does not happen every time.

Prompt 4: What does `pop(k)` do?

Answer:

- `pop(k)` removes the element at index `k`
- All elements **after index k** must be shifted one position to the left

Example:

```
A = [A, B, C, D, E]

A.pop(1)   # removes B
```

Result:

```
[A, C, D, E]
```

Prompt 5: Why does `pop(k)` take O(n − k) time?

Answer:

Because:

- Elements at positions `k+1` through `n-1` must be shifted left
- Number of shifts = `n - k - 1`

Time Complexity:

$$O(n-k)$$

Prompt 6: Why is `pop(0)` the most expensive removal?

Answer:

- Removing the first element shifts **every other element**
- Number of shifts ≈ `n - 1`

Time Complexity:

$$\Omega(n)$$

Prompt 7: Visual illustration of `pop(k)`

Answer (ASCII Diagram – Figure 5.17 style):

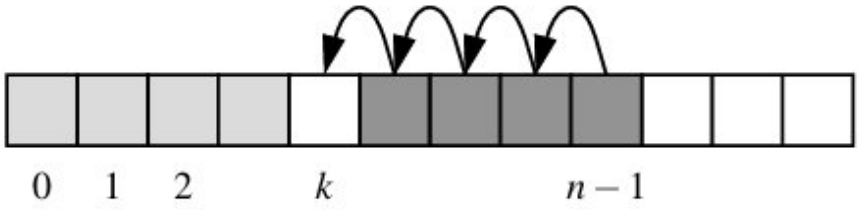

Figure 5.17: Removing an element at index k of a dynamic array.

Prompt 8: What does `remove(value)` do?

Answer:

- Removes the **first occurrence** of `value`
- Raises `ValueError` if the value is not found
- Does **not** take an index as input

Example:

```
A = [3, 5, 7, 5, 9]

A.remove(5)
```

Result:

```
[3, 7, 5, 9]
```

Prompt 9: How does `remove(value)` work internally?

Answer:

1. **Search phase:**
   - Scan from index `0` until the value is found → O(n)
2. **Shift phase:**
   - Shift elements left to fill the gap → O(n − k)

Both phases are linear.

Prompt 10: Why is there no "efficient" case for `remove`?

Answer:

Even in the **best case**:

- The value is found at index `0`
- All remaining elements must still be shifted

Prompt 11: What is the time complexity of `remove(value)`?

Answer:

Case	Time Complexity
Best case	Ω(n)
Worst case	O(n)
Average case	O(n)

There is **no O(1) case**.

Prompt 12: What does Code Fragment 5.6 demonstrate?

Answer:

It shows a **manual implementation** of `remove(value)` using a dynamic array:

```
for k in range(self.n):              # search
    if self.A[k] == value:
        for j in range(k, self.n-1): # shift
            self.A[j] = self.A[j+1]
        self.A[self.n-1] = None
```

```
        self.n -= 1
        return
raise ValueError("value not found")
```

Prompt 13: Visual illustration of `remove(value)`

Answer (ASCII Diagram):

```
Array:
[A, B, C, D, E]
remove(C)
Search:
A → B → C (found)
Shift:
D → index 2
E → index 3
Result:
[A, B, D, E]
```

Prompt 14: Summary Table

Answer:

Method	Removes	Time Complexity
`pop()`	Last element	O(1) amortized
`pop(k)`	Element at index k	O(n − k)
`remove(value)`	First matching value	O(n)

The Insertion-Sort Algorithm

Prompt 1: What is insertion-sort?

Answer:

Insertion-sort is a **simple comparison-based sorting algorithm** that builds the final sorted array **one element at a time**.

It works the same way you might sort playing cards in your hand:

- Take one card
- Insert it into its correct position among the already-sorted cards

Prompt 2: What is the main idea behind insertion-sort?

Answer:

- The left part of the array is always kept **sorted**
- At each step, a new element is **inserted into its correct position** in that sorted portion

At iteration `k`:

- Elements `A[0]` through `A[k-1]` are already sorted
- Element `A[k]` is inserted into its proper position

Prompt 3: What are the steps of insertion-sort (high-level)?

Answer:

1. Start with the second element (index 1)
2. Compare it with elements to its left
3. Shift larger elements one position to the right
4. Insert the element in the correct position
5. Repeat until the entire array is sorted

Prompt 4: What is the pseudocode for insertion-sort?

Answer (Code Fragment 5.9):

```
Algorithm InsertionSort(A):
```

```
Input: An array A of n comparable elements

Output: A sorted in nondecreasing order

for k from 1 to n − 1 do

    Insert A[k] into its proper location within A[0], A[1], ..., A[k−1]
```

Prompt 5: How does the Python implementation work?

Answer (Code Fragment 5.10 Explained):

```
def insertion_sort(A):

    for k in range(1, len(A)):

        cur = A[k]        # current element

        j = k

        while j > 0 and A[j-1] > cur:

            A[j] = A[j-1] # shift element right

            j -= 1

        A[j] = cur        # insert cur in correct position
```

Key Variables:

- `cur`: element being inserted
- `j`: index used to shift elements leftward
- `A[0...k-1]`: already sorted subarray

Prompt 6: How does insertion-sort work step by step?

Answer (Example):

Initial array:

```
[ C, B, D, A, E ]
```

Step-by-step execution:

```
Step 1: [ C | B D A E ]

Insert B → shift C

→ [ B C | D A E ]
```

```
Step 2: [ B C | D A E ]

D already in place

→ [ B C D | A E ]

Step 3: [ B C D | A E ]

Insert A → shift D, C, B

→ [ A B C D | E ]

Step 4: [ A B C D | E ]

E already in place

→ [ A B C D E ]
```

✔ Sorted!

Prompt 7: Visual illustration of insertion-sort

Answer (ASCII Diagram):

```
Iteration k = 3

-----------------------

Sorted part | Unsorted

[A B C]     | D

Insert D → no move

Iteration k = 4

-----------------------

[A B C D] | E

Insert E → no move
```

Another example with shifts:

```
Insert A into [B C D]

Shift D → C → B

Result:

[A B C D]
```

Prompt 8: What is the running time of insertion-sort?

Answer:

Case	Running Time	Reason
Best case	**O(n)**	Already sorted, no shifts
Average case	**O(n²)**	About half shifts per element
Worst case	**O(n²)**	Reverse order, maximum shifts

Worst-case explanation:

- For each element, you may need to shift up to `k` elements
- Total comparisons ≈
- $1 + 2 + 3 + \cdots + (n-1) = O(n^2)$

Prompt 9: When is insertion-sort efficient?

Answer:

Insertion-sort is efficient when:

- The array is **small**
- The array is **nearly sorted**
- You want a **stable** sort
- Memory usage must be minimal (in-place)